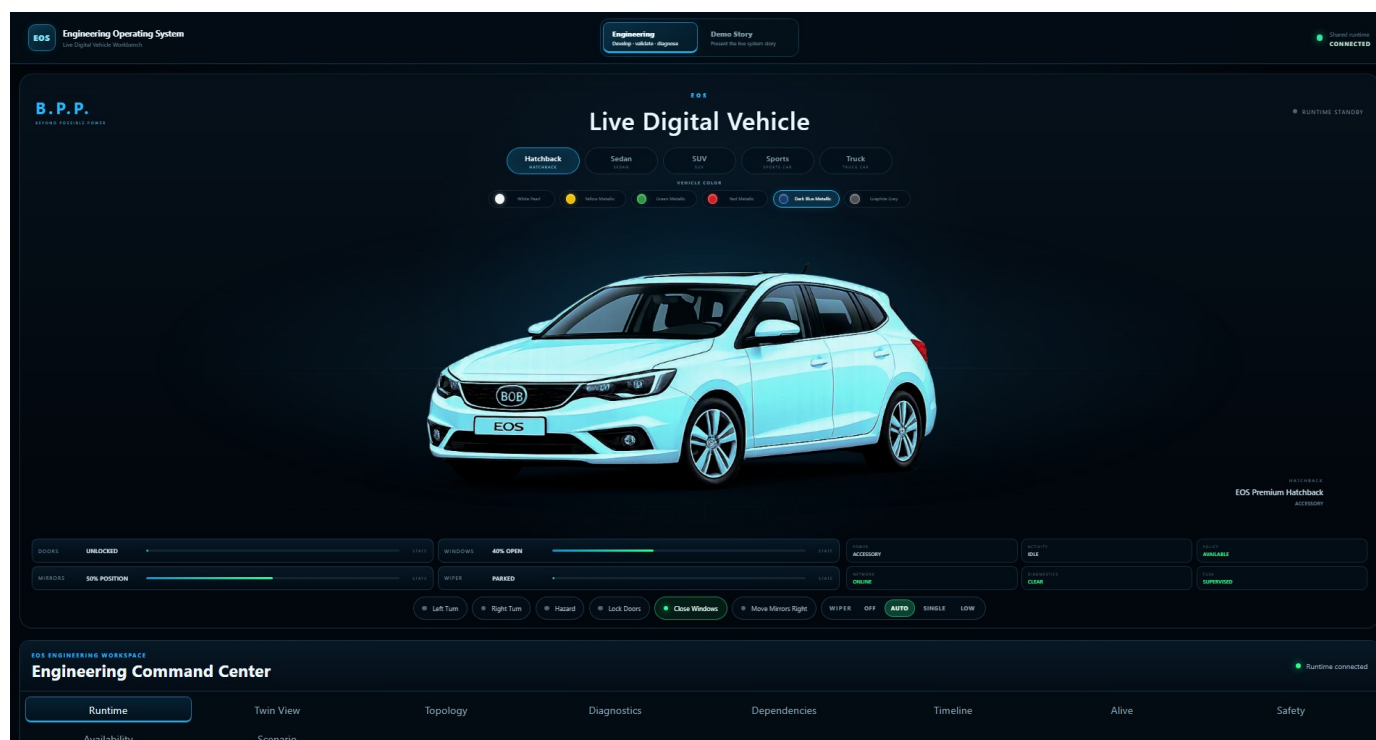


BCM Emulator V1.0

QUICK START

From first launch to a meaningful SDV engineering experiment in about five minutes.



Engineering workspace - Live Digital Vehicle and Engineering Command Center on one shared runtime.

PUBLIC PREVIEW

bcm-emulator.netlify.app

Operate • Validate • Observe

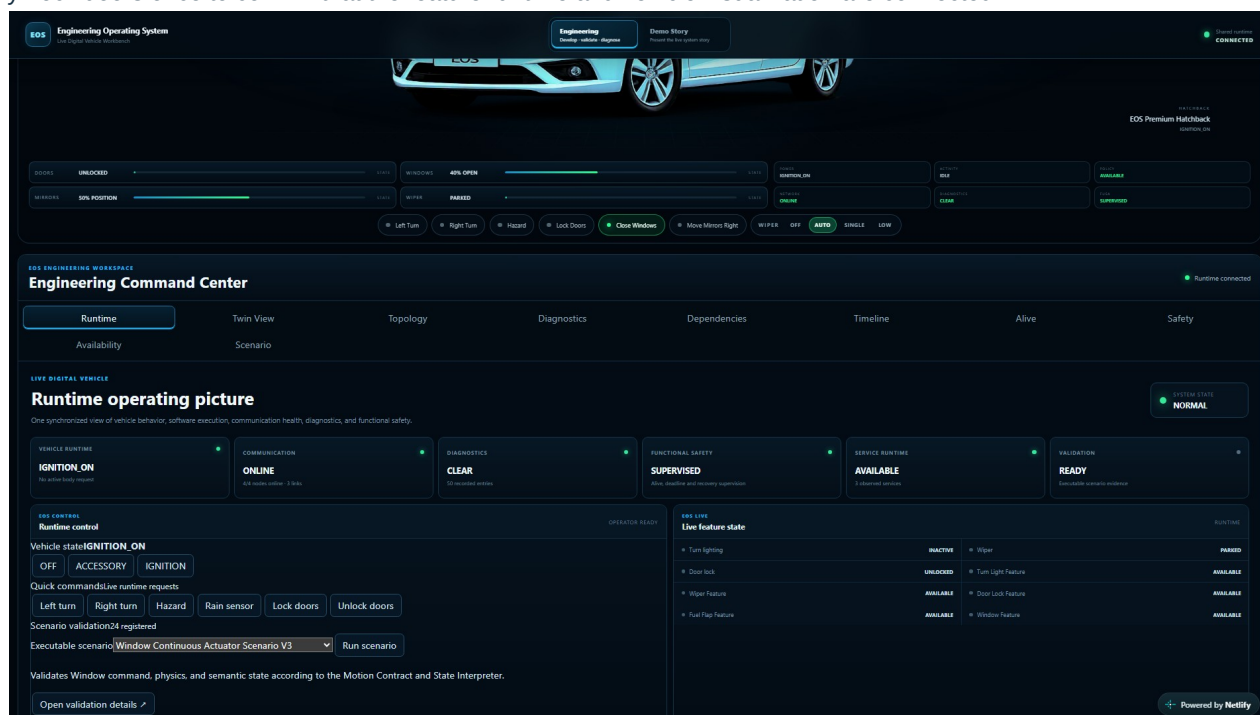


[Open BCM Emulator V1.0](https://bcm-emulator.netlify.app)

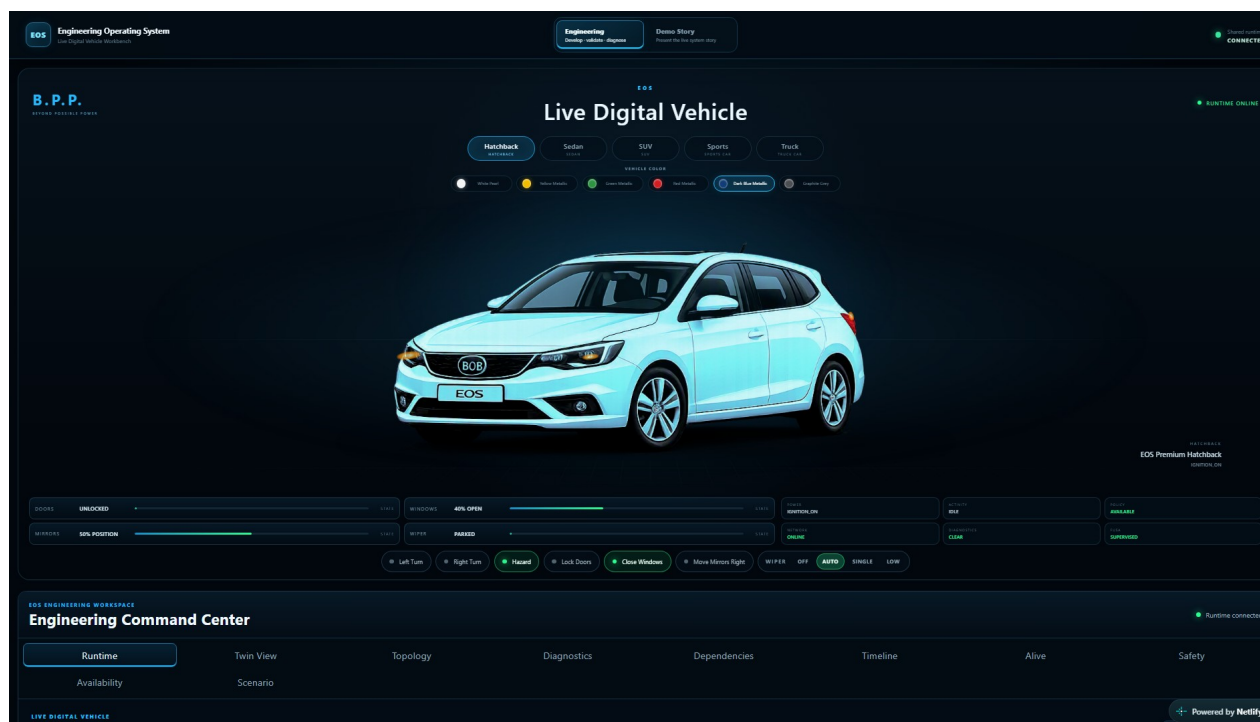
1. Open the Emulator and Bring the Vehicle Online

Start with one simple interaction. You do not need to understand the complete architecture first.

- 1 Open bcm-emulator.netlify.app and stay in the Engineering workspace.
- 2 Open Runtime and set Vehicle state to IGNITION.
- 3 Toggle Hazard and observe the signaling behavior on the Live Digital Vehicle.
- 4 Try Lock doors once to confirm that the feature runtime and vehicle visualization are connected.



IGNITION selected in Runtime Control.



Hazard active on the Live Digital Vehicle.

What this proves: The hosted UI, feature runtime and vehicle visualization are connected to the same running BCM system.

2. Run Your First Engineering Scenario

Scenarios establish a known baseline automatically, execute a repeatable test and leave the result visible for inspection.

- 1 In Engineering -> Runtime, locate Scenario validation.
- 2 Select DoorLock Discrete Actuator Scenario V1.
- 3 Select Run scenario.
- 4 Open Scenario / validation details and confirm the final result is PASS.

The screenshot shows the EOS Engineering Operating System interface. The 'Engineering' tab is active, and the 'Runtime' sub-tab is selected. A dropdown menu is open, showing a list of scenarios, with 'DoorLock Discrete Actuator Scenario V1' highlighted. The main dashboard displays several status indicators: 'DIAGNOSTICS' is 'CLEAR', 'FUNCTIONAL SAFETY' is 'SUPERVISED', 'SERVICE RUNTIME' is 'AVAILABLE', and 'VALIDATION' is 'READY'. Below these, there's a 'Live feature state' table with columns for Feature, State, and Runtime. The table lists features like Turn Lighting, Door lock, Wiper Feature, and Fuel Tap Feature, with their current states (e.g., INACTIVE, UNLOCKED, AVAILABLE) and runtime status (e.g., PARKED, AVAILABLE). At the bottom, there's an 'Event timeline' section showing a list of events with timestamps and descriptions.

DoorLock scenario selected.

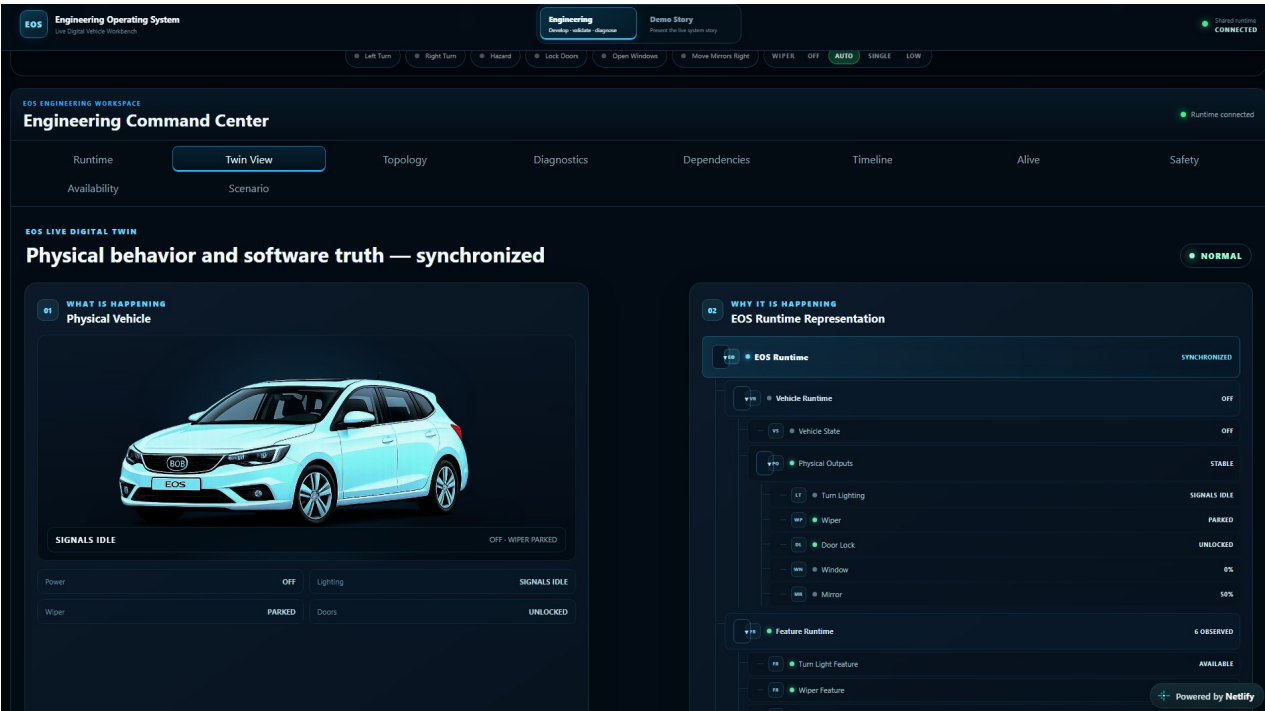
The screenshot shows the EOS Engineering Operating System interface with the 'Runtime Reset Reconstruction Validation V1' scenario selected. The interface displays a detailed description of the scenario, its purpose, and a table of execution results. The results show a 'PASS' status with a duration of 825 ms and a status of 'IDLE'. Below the results, there's an 'Execution Timeline' table with columns for #, Description, Category, Status, Duration, and Timestamp. The table lists 7 events, including 'Initial state shall be UNLOCKED', 'Request DOOR LOCK', 'DoorLock shall enter LOCKING', 'DoorLock shall become LOCKED', 'Request DOOR UNLOCK', 'DoorLock shall enter UNLOCKING', and 'DoorLock shall become UNLOCKED'. All events have a status of 'PASSED'.

Successful execution with PASS and the step-by-step execution timeline.

Reset behavior: You do not need a manual "make everything healthy" button. The Scenario Runner establishes the canonical runtime baseline before execution.

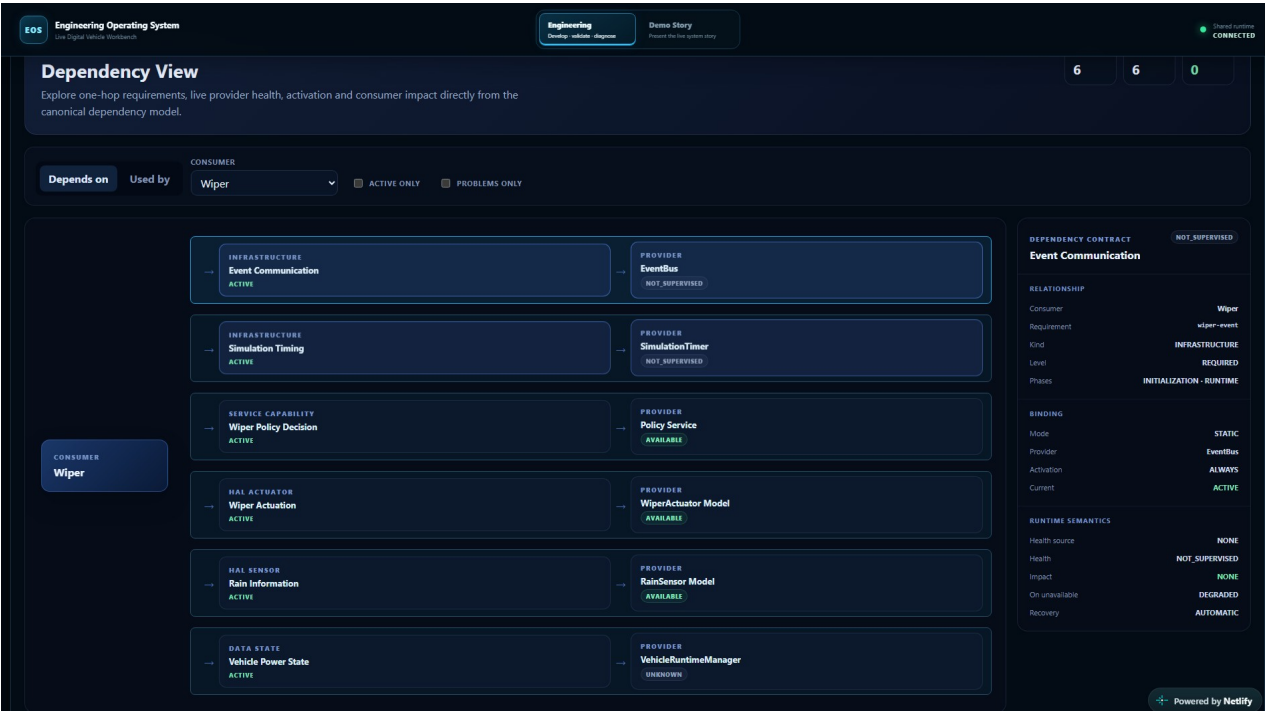
3. Look Behind the Vehicle Behavior

The vehicle is only one view. Use engineering views to see why the visible behavior is happening.



Twin View - physical behavior and software truth synchronized.

Twin View: correlate what the vehicle is doing with the runtime representation.



Dependency View - consumer, requirement, provider, health and impact.

Dependencies: see what a consumer needs, which provider supplies it and how provider health affects the consumer.

Remember: AVAILABLE does not mean ACTIVE, and INACTIVE does not mean unhealthy. Domain state, capability state and operational state answer different questions.

4. Reconstruct What Happened

The Event Timeline is historical evidence: it shows order, trigger, reason, participants and payload.



Time	Event	Payload	Trigger	Reason	Feature	Source	Target
4:10:35 PM	COMMUNICATION_MESSAGE_COMPLETED	["messageId":"COMM-1790431834648-1740","event":"SET_DOOR_LOCK","sourceNode":"Front Zone Controller","targetNode":"Front Zone Controller","completedAtMs":1790431835177]				Front Zone Controller	Front Zone Controller
4:10:35 PM	COMMUNICATION_HOP_COMPLETED	["messageId":"COMM-1790431834648-1740","event":"SET_DOOR_LOCK","technology":{"linkProtocol":"LIN"},"source":"Front Zone Controller","target":"DoorLockActuatorModel","label":"Actuator bus delivery","hopIndex":0,"hopCount":1,"state":"COMPLETED","startedAtMs":1790431835177,"durationMs":520]				Front Zone Controller	Front Zone Controller
4:10:34 PM	FEATURE_OPERATIONAL_STATE_CHANGED	["feature":"DoorLockFeature","operationalState":"INACTIVE","trigger":"domainUNLOCKED","source":"FeatureLifecycleCoordinator"]	domainUNLOCKED		DoorLockFeature	FeatureLifecycleCoordinator	
4:10:34 PM	DOORLOCK_STATE_CHANGED	["feature":"DoorLockFeature","oldState":"UNLOCKING","newState":"UNLOCKED"]			DoorLockFeature		
4:10:34 PM	DOORLOCK_ACTUATOR_STATE_CHANGED	["currentState":"IDLE","transitionElapsedMs":300,"transitionDurationMs":300]					
4:10:34 PM	COMMUNICATION_MESSAGE_COMPLETED	["messageId":"COMM-1790431834284-1739","event":"SET_REAR_LAMP","sourceNode":"Front Zone Controller","targetNode":"RearZoneController","completedAtMs":1790431834818]				Front Zone Controller	RearZoneController
4:10:34 PM	COMMUNICATION_HOP_COMPLETED	["messageId":"COMM-1790431834284-1739","event":"SET_REAR_LAMP","technology":{"linkProtocol":"LIN"},"source":"Front Zone Controller","target":"LampActuatorModel","label":"Actuator bus delivery","hopIndex":0,"hopCount":1,"state":"COMPLETED","startedAtMs":1790431834818,"durationMs":520]				Front Zone Controller	RearZoneController
4:10:34 PM	COMMUNICATION_MESSAGE_COMPLETED	["messageId":"COMM-1790431834284-1737","event":"SET_LAMP","sourceNode":"Front Zone Controller","targetNode":"Front Zone Controller","completedAtMs":1790431834818]				Front Zone Controller	Front Zone Controller
4:10:34 PM	COMMUNICATION_HOP_COMPLETED	["messageId":"COMM-1790431834284-1737","event":"SET_LAMP","technology":{"linkProtocol":"LIN"},"source":"Front Zone Controller","targetNode":"Front Zone Controller","completedAtMs":1790431834818]				Front Zone Controller	Front Zone Controller

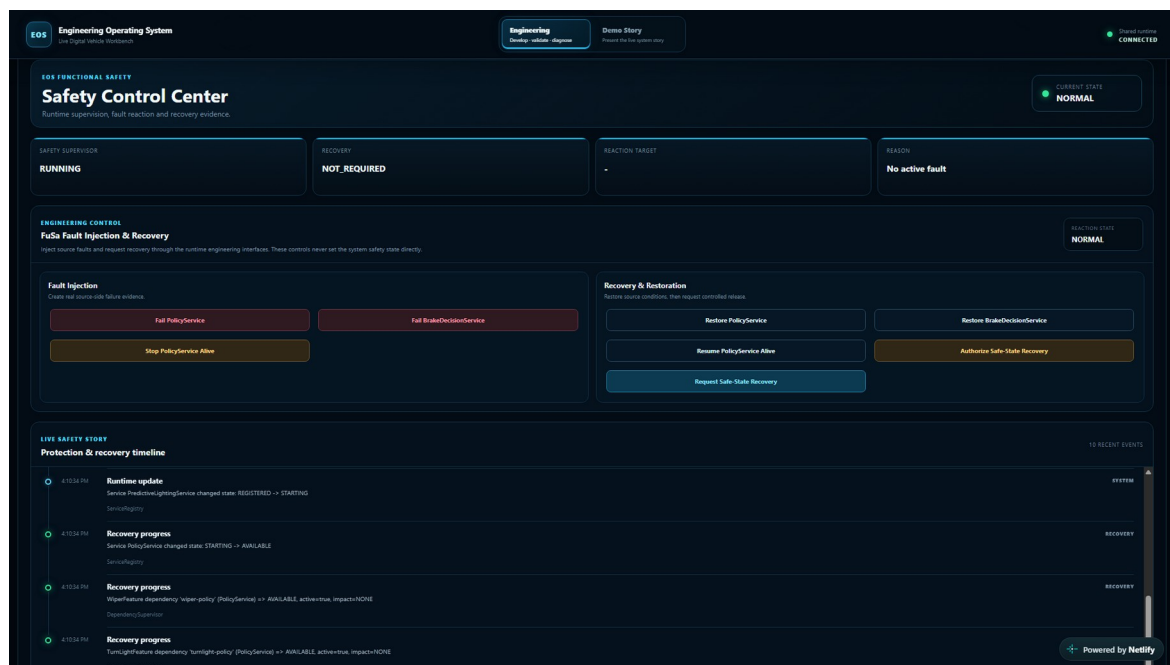
Event Timeline - chronological engineering evidence.

History ≠ current state

A past fault may remain visible after the current runtime has already recovered.

5. Understand the Functional Safety View

The public V1.0 system-level safety states are NORMAL, DEGRADED and SAFE STATE.



EOS FUNCTIONAL SAFETY

Safety Control Center

Runtime supervision, fault reaction and recovery evidence.

CURRENT STATE
NORMAL

SAFETY SUPERVISION
RUNNING

RECOVERY
NOT_REQUIRED

REACTION TARGET
-

REASON
No active fault

ENGINEERING CONTEXT
Fault Injection & Recovery
Inject runtime faults and request recovery through the runtime engineering interfaces. These controls never set the system safety state directly.

Fault Injection
Create new runtime-side failure evidence.

Recovery & Restoration
Recover safety conditions, then request controlled release.

LIVE SAFETY STORY
Protection & recovery timeline

18 RECENT EVENTS

- 4:10:34 PM **Runtime update**
Service PredictiveLightingService changed state REGISTERED => STARTING
ServiceLighting
- 4:10:34 PM **Recovery progress**
Service PolicyService changed state STARTING => AVAILABLE
ServicePolicy
- 4:10:34 PM **Recovery progress**
WiperFeature dependency Wiper-policy (PolicyService) => AVAILABLE, achievement: impact:NONE
DependencyWiper
- 4:10:34 PM **Recovery progress**
TurnSignalFeature dependency TurnSignal-policy (PolicyService) => AVAILABLE, achievement: impact:NONE
DependencyTurnSignal

Safety Control Center in the NORMAL baseline.

Guided experiments first: For a first safety experiment, use a predefined safety/recovery scenario instead of combining arbitrary fault-injection controls.

6. Follow a Fault and Recovery Across Several Kinds of Evidence

Read state, safety evidence, diagnostics and event history together.

1 SAFE STATE active

2 Safety evidence

3 Recovery diagnostics

4 Event evidence

One recovery experiment seen through four complementary evidence views.

Read the sequence as: SAFE STATE active -> safety evidence -> recovery diagnostics -> event evidence.

Key engineering idea: Technical recovery of the failed component does not automatically authorize the system to leave SAFE STATE. Recovery requires valid technical evidence and controlled release.

7. Run the EOS Integrated Mission

The strongest single end-to-end demonstration of vehicle functions, distributed runtime behavior, fault handling, FuSa and recovery.

EOS Integrated Mission

Flagship SDV showcase for vehicle functions, distributed communication, fault handling, FuSa and recovery. Runs Acts I-IV, including signaling, automatic wiper operation, HPC fault injection, diagnostics, FuSa supervision and recovery.

EOS

Integrated

Mission

SDV

Presentation

WakeUp

DailyOperation

DoorLock

Mirror

Window

Lighting

DrivingEnvironment

Signaling

HazardArbitration

RainSensor

Wiper

EngineeringExcellence

FaultInjection

Diagnostics

HPCRecovery

Resilience

Run

Result: **PASS**

Duration: 77853 ms

Status: IDLE

Execution Timeline

#	Description	Category	Status	Duration	Timestamp
1	ACT I - Vehicle power ON	ACTION	PASSED	0 ms	4:24:55 PM
2	ACT I - Core compute nodes online	ACTION	PASSED	0 ms	4:24:56 PM
3	ACT I - Ethernet, CAN FD, SOME/IP and LIN ready	ACTION	PASSED	1346 ms	4:24:57 PM
4	ACT I - Vehicle READY	ACTION	PASSED	2227 ms	4:24:59 PM
5	ACT II - Prepare secured vehicle	ACTION	PASSED	488 ms	4:25:03 PM
6	ACT II - Door unlock	ACTION	PASSED	501 ms	4:25:04 PM
7	ACT II - Mirror unfold	ACTION	PASSED	965 ms	4:25:05 PM
8	ACT II - Window partially open	ACTION	PASSED	621 ms	4:25:07 PM
9	ACT II - Welcome lighting	ACTION	PASSED	1327 ms	4:25:09 PM
10	ACT III - Left indicator requested	ACTION	PASSED	2533 ms	4:25:12 PM
11	ACT III - Hazard takes priority	ACTION	PASSED	1984 ms	4:25:15 PM
12	ACT III - Hazard released — left indicator resumes	ACTION	PASSED	1053 ms	4:25:18 PM
13	ACT III - Left indicator completed	ACTION	PASSED	833 ms	4:25:20 PM
14	ACT III - Automatic wiper armed	ACTION	PASSED	763 ms	4:25:23 PM
15	ACT III - Rain detected	ACTION	PASSED	767 ms	4:25:24 PM
16	ACT III - Automatic wiper operating	ACTION	PASSED	3488 ms	4:25:26 PM
17	ACT III - Rain cleared — wiper parks	ACTION	PASSED	1943 ms	4:25:30 PM
18	ACT IV - Distributed system healthy	ACTION	PASSED	3487 ms	4:25:34 PM
19	ACT IV - Inject Central HPC fault	ACTION	PASSED	2620 ms	4:25:38 PM
20	ACT IV - Diagnostics detect communication impact	ACTION	PASSED	3002 ms	4:25:43 PM
21	ACT IV - FuSa supervision enters DEGRADED state	ACTION	PASSED	3232 ms	4:25:48 PM
22	ACT IV - Recover Central HPC	ACTION	PASSED	2507 ms	4:25:52 PM
23	ACT IV - Central HPC online — communication restored	ACTION	PASSED	2725 ms	4:25:56 PM
24	ACT IV - FuSa supervision returns to NORMAL	ACTION	PASSED	2938 ms	4:26:01 PM
25	ACT IV - System recovered — vehicle READY	ACTION	PASSED	3240 ms	4:26:06 PM
26	CLEANUP - Restore initial vehicle state	ACTION	PASSED	1573 ms	4:26:10 PM

EOS Integrated Mission completed successfully - Acts I-IV visible in the execution timeline.

The diagram illustrates the vehicle's network topology from a top-down perspective. A central car model is connected to various control units: ADAS HPC, Central HPC, Front Zone Controller, and Rear Zone Controller. These units are interconnected via AUTOMOTIVE ETHERNET, SOME/IP, and CAN FD BACKBONE. A SECURE ROUTE is highlighted between the Front and Rear Zone Controllers. LIN buses are shown at the front and rear. A status indicator at the bottom shows 'LEFT INDICATOR ACTIVE'. The interface includes a 'Left Indicator ON' control at the top left and a 'Powered by Nuttix' logo at the bottom right.

While it runs, the same shared runtime can be observed through the live topology / presentation view.

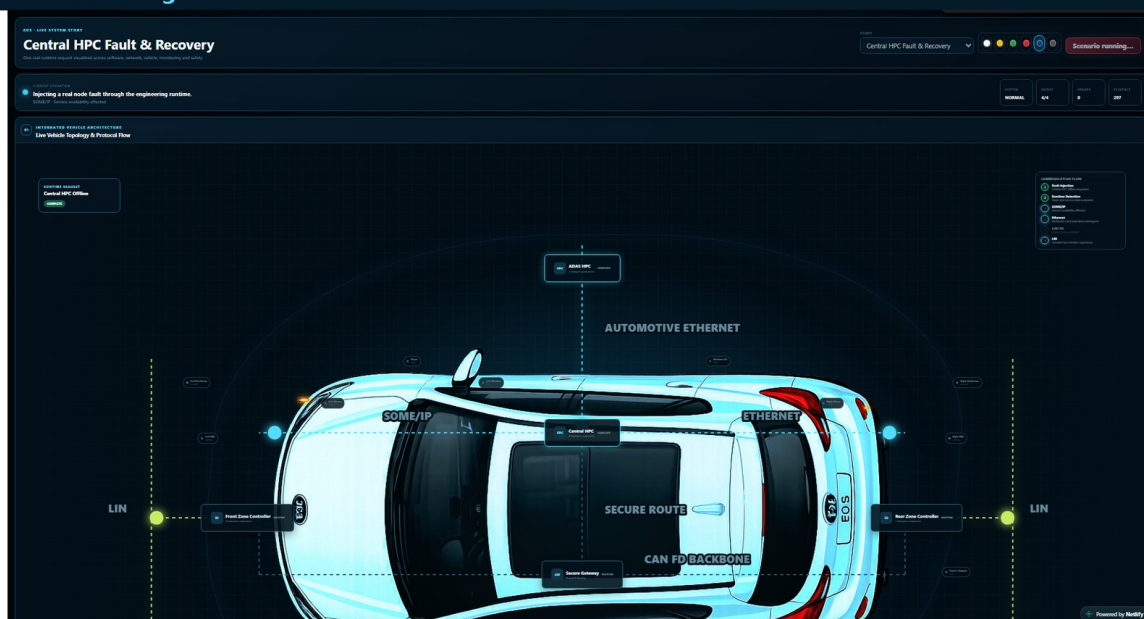
Best single overview: If you only have time for one advanced scenario, use EOS Integrated Mission and move between Runtime, Twin View, Topology, Dependencies, Safety and Scenario while it runs.

BCM Emulator V1.0 • Quick Start • Public Preview

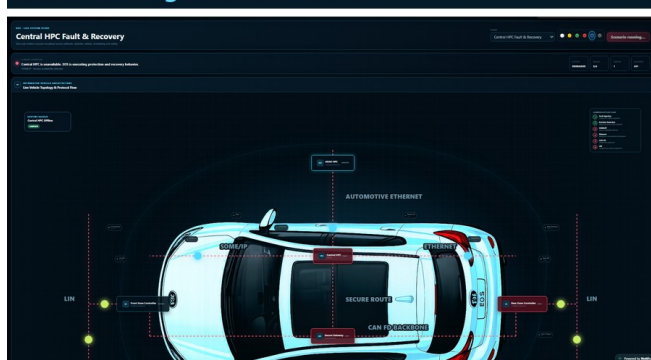
8. Switch to Demo Story

Demo Story uses the same shared runtime but presents it as an end-to-end live story.

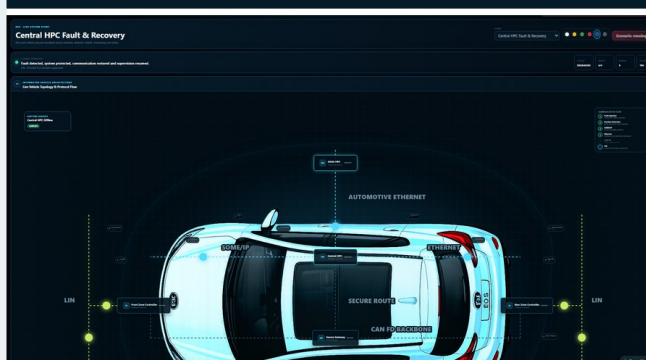
1 Scenario running



2 Fault / degraded behavior



3 Communication restored



Central HPC Fault & Recovery - running, fault/degraded behavior and communication recovery.

V1.0 Demo Stories: Left Indicator Execution and Central HPC Fault & Recovery. **Engineering** is for detailed diagnosis; **Demo Story** is for presenting the complete runtime story quickly.

WHERE TO GO NEXT

UI: Section 4
Features: Section 6
States: Section 7
Faults & recovery: Section 8

DEEPER ENGINEERING

Scenarios: Sections 9-12
Observability: Section 15
Reset/reconstruction: Section 16
Full User Guide: BCM Emulator V1.0

THE FIVE-MINUTE PATH

Open → **IGNITION** → **Hazard / Door Lock** → **First scenario** → **PASS** → **Twin / Dependencies** → **Recovery** → **Integrated Mission**

Operate -> Validate -> Observe That is the fastest way to understand what BCM Emulator V1.0 is about.

Ready to try it? bcm-emulator.netlify.app